

Formal Verification Intro

with SystemVerilog Assertions (SVAs)

From executable design intent to property checking

Simulation-based verification

Constrained random

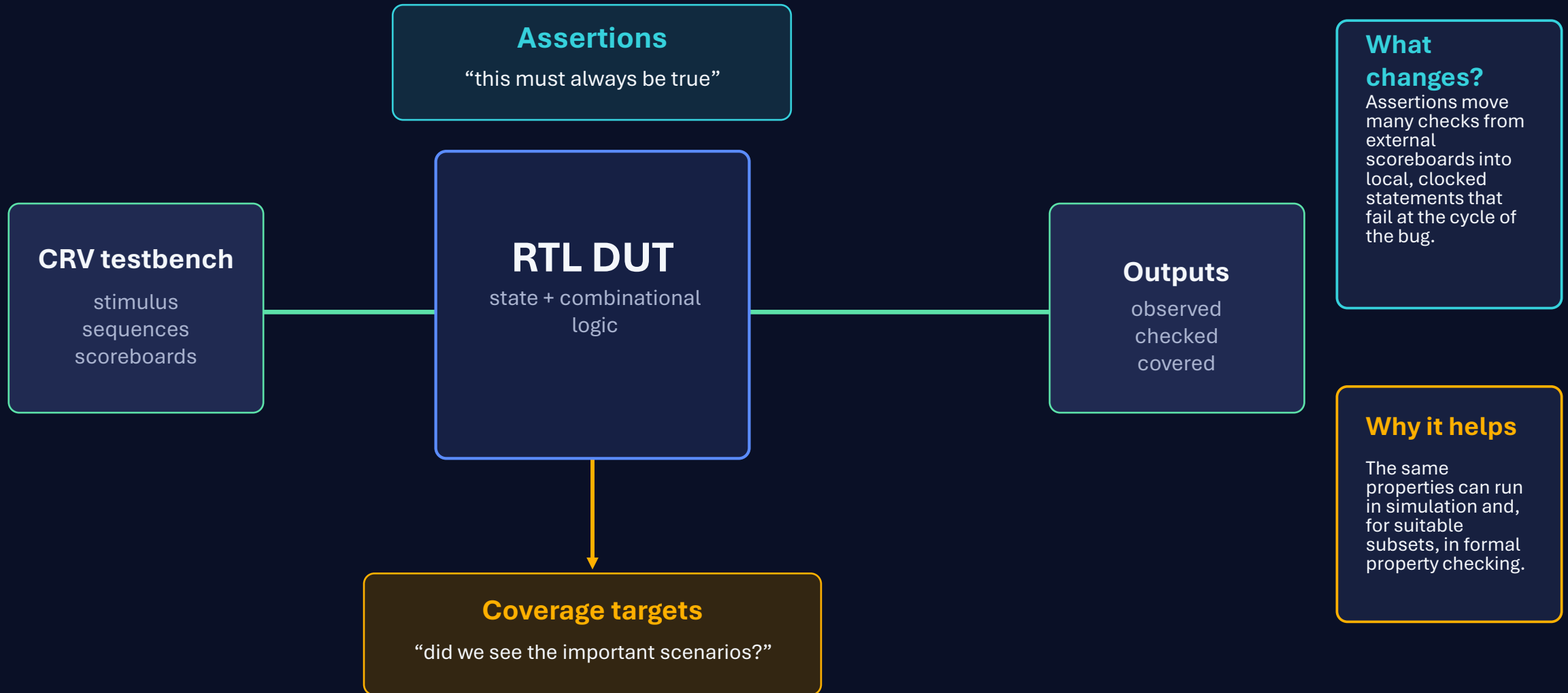
Formal property checking

Goal: understand where assertions fit, how they are written, and why formal tools can cover more than random tests alone.

Functional verification of RTL designs

Why assertion-based verification?

Assertions are executable contracts placed close to the design behavior.



Assertions are contracts over time

SVAs describe legal behavior — not the test that produces it.

Immediate assertions

Procedural checks evaluated now. Good for local combinational or algorithmic assumptions.

Concurrent assertions

Clocked temporal properties. They observe signal history and future obligations.

Assume / cover

Formal adds assumptions to constrain inputs and cover properties to find interesting traces.

Design intent



SVA property



Checker



Fail / Pass /
Cover

Example intent: “Once a request is accepted, an acknowledgement must arrive within 1–3 cycles.”

Anatomy of a simple SVA

A property binds a clock, reset behavior, an antecedent, and a consequent.

```
property p_req_ack;  
  @(posedge clk) disable iff (!rst_n)  
    req |-> ##[1:3] ack;  
endproperty
```

```
a_req_ack: assert property  
(p_req_ack);
```

@(posedge clk)

Sampling event: the property only sees values at clock ticks.

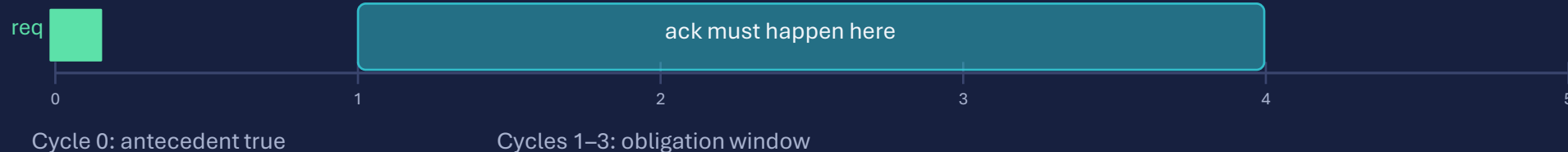
disable iff

Reset/abort condition: property is inactive during reset.

|-> ##[1:3]

If req holds, ack must hold one to three cycles later.

Temporal view



SVA vocabulary: the small set you need first

Start with a few operators; add advanced sequences later.

| ->

overlapped implication

consequent starts same cycle

| =>

non-overlapped implication

consequent starts next cycle

##N

cycle delay

wait N sampled clocks

##[m:n]

bounded delay

wait m to n clocks

**\$stable(x
)**

data stability

x unchanged from previous sample

Mental model

Every clock edge starts a tiny watcher.
If the antecedent is true, the watcher tracks the promised future behavior.



do not overfit to one test

State space: simulation samples, formal explores

This is the key intuition behind property checking.

Simulation + CRV

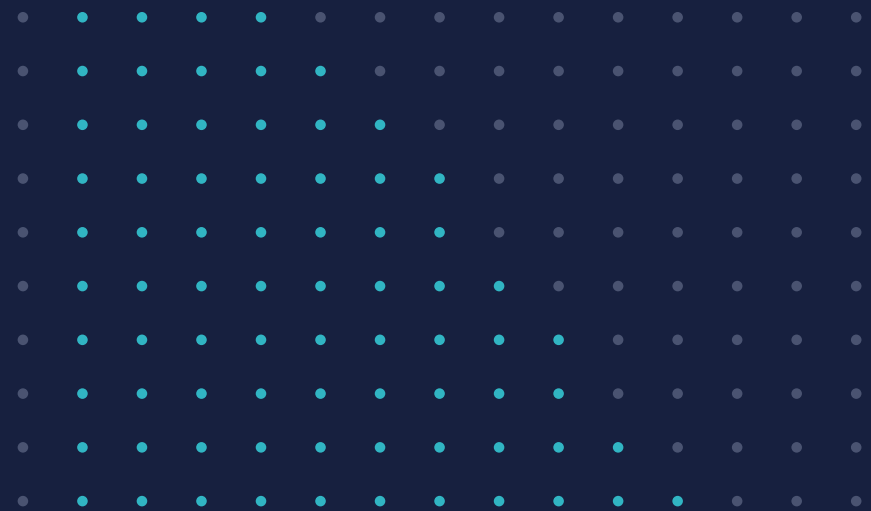
Many randomized traces
through an enormous state space



Good at finding bugs along executed paths.

Formal property checking

Mathematically searches all reachable states
up to a bound, or proves by induction

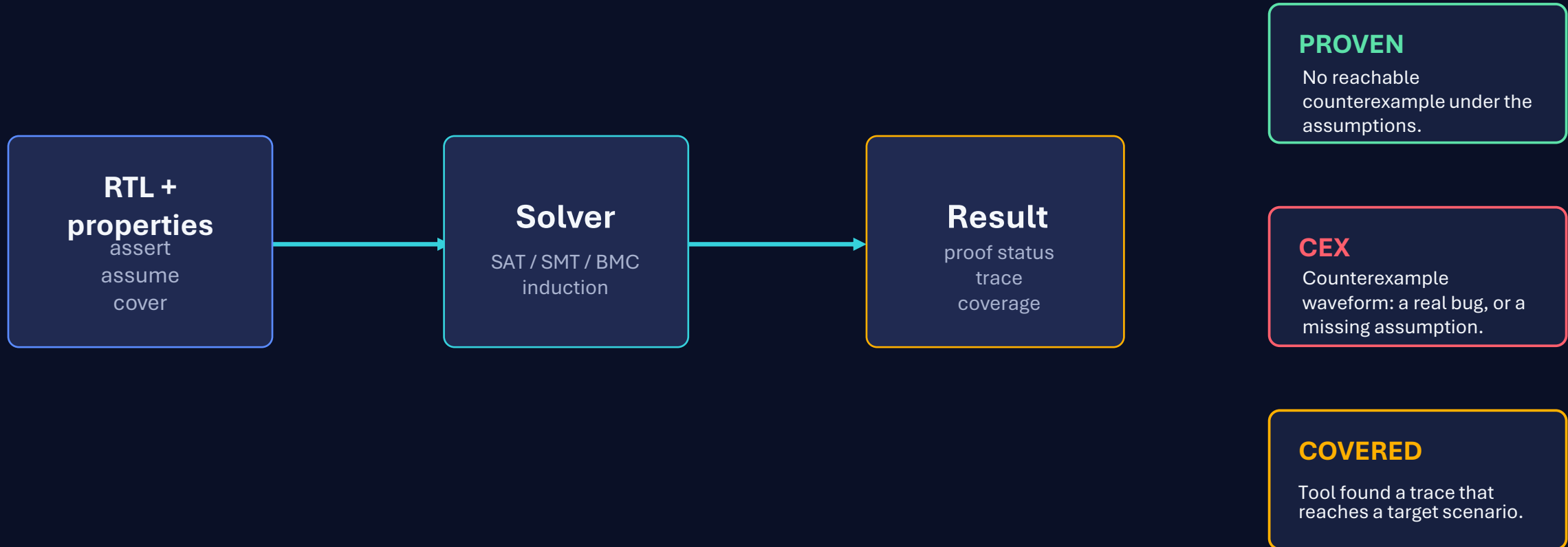


reachable state space

Good at proving absence of a property violation — within
assumptions and tool capacity.

Formal property checking in one slide

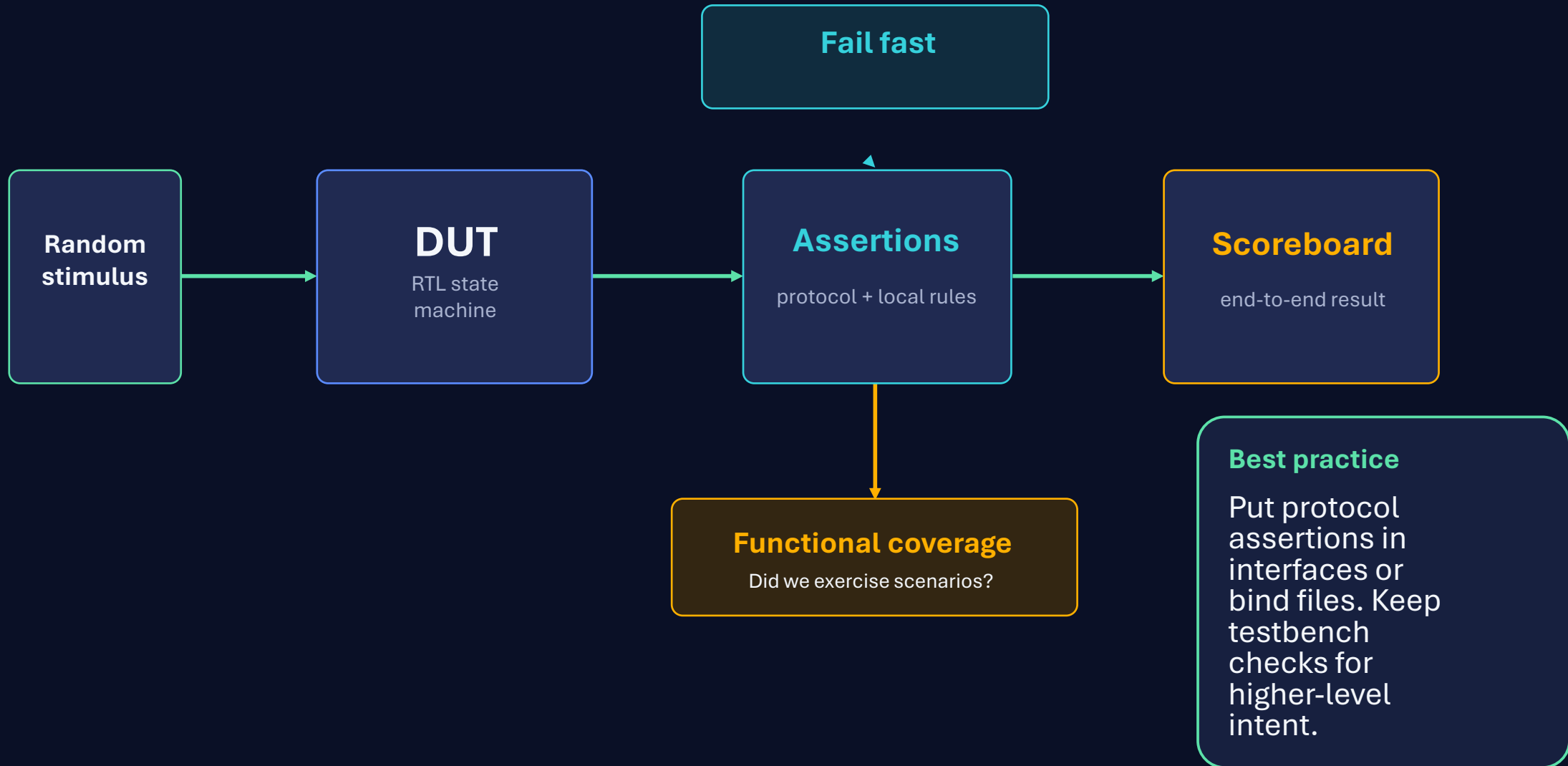
Formal tools search traces automatically; you provide the design, assumptions, and properties.



Bounded model checking answers “is there a violation in N cycles?” · Induction tries to prove the property for all cycles.

How assertions fit into a CRV course flow

Use assertions as complementary checkers, not a replacement for testbenches.



Useful assertion patterns and common pitfalls

Good SVAs are small, local, and tied to design intent.

Mutual exclusion `assert !(rd && wr);`

Request → response `req |-> ##[1:5] gnt;`

Stable while stalled `valid && !ready | =>
$stable(data);`

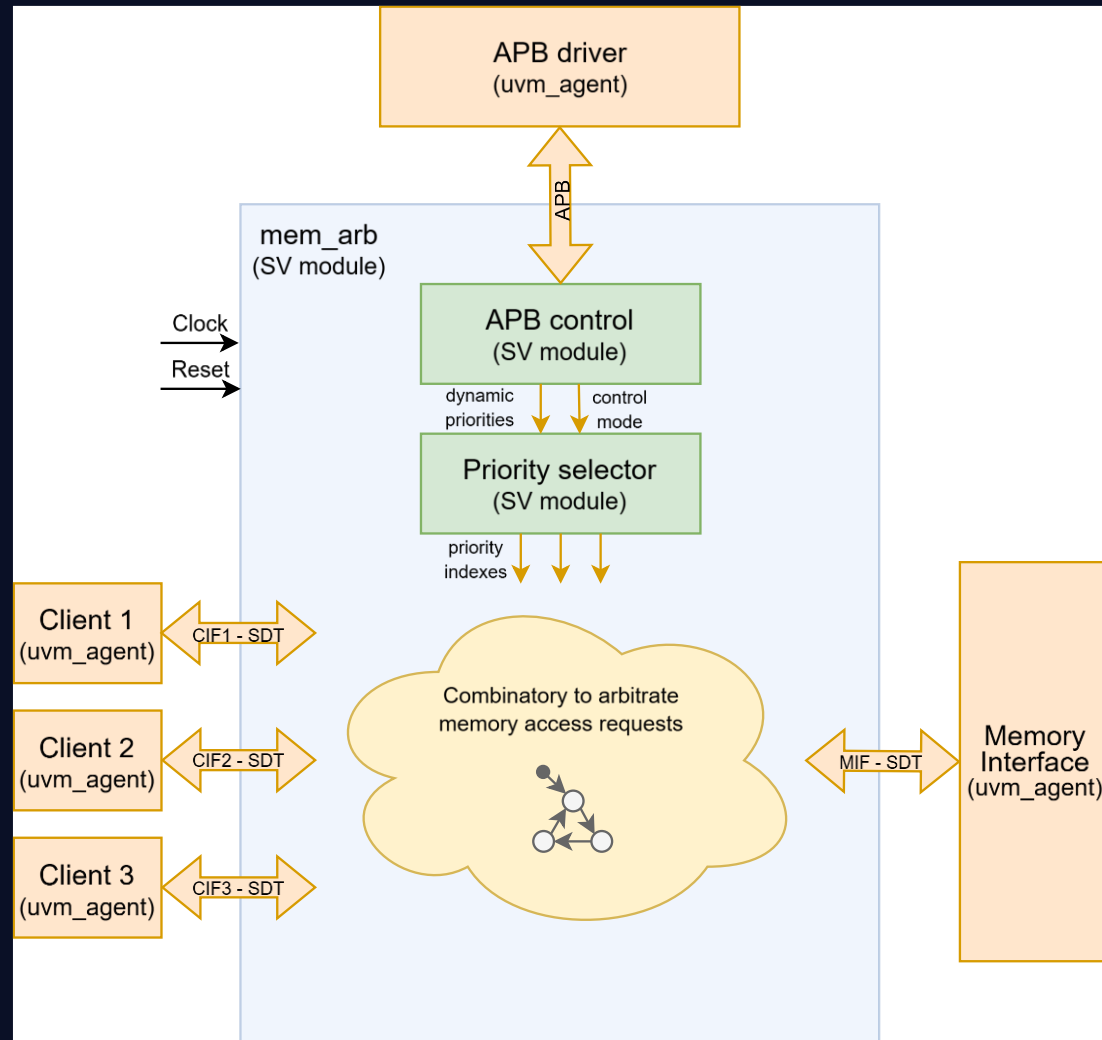
No X after reset `!$isunknown(control);`

Pitfalls

- No reset handling → false failures during initialization
- Weak antecedent → property almost never triggers
- Missing assumptions → formal explores illegal environments
- Overly broad property → hard to debug and hard to prove
- Vacuous pass → proof succeeds because premise is unreachable

Rule of thumb: one assertion should explain one design rule in one sentence.

Device Under Test: Memory Arbiter (MARB)



Device Under Test: Implementation

- 3 client interfaces (CIF)
- 1 memory interface (MIF)
- 1 APB interface (APB)
- Serves the client with the highest priority
- Default priority:
 - $\text{CIF1} > \text{CIF2} > \text{CIF3}$
- All interfaces follow SDT protocol

SyoSil Data Transfer Protocol

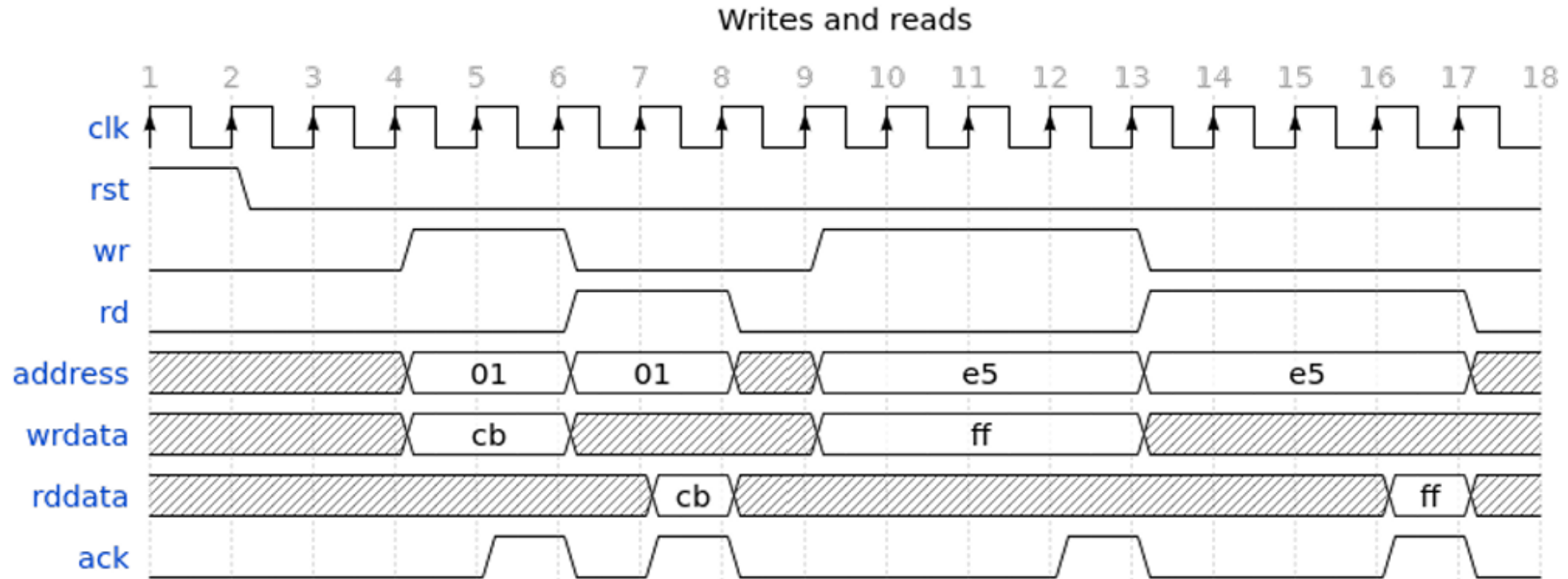


Fig. 3.1: Write and read operations using the *SDT* protocol.